TINAS: Transformer Interaction among N Agents with Shared Parameters

Kasra Fouladi
Iran University of Science and Technology (IUST)
k4sr405@gmail.com
https://github.com/kasrafouladi/StrikeForce-No-BPTT-via-FAP

Repository: https://github.com/kasrafouladi/StrikeForce-No-BPTT-via-FAP

Abstract

We study the problem of learning inter-agent communication in multi-agent reinforcement learning (MARL) with parameter sharing. Standard PPO treats each agent's loss as local: no mechanism lets agents learn to communicate in a gradient-aware way. We propose TINAS, a framework in which each agent emits a message to every other agent at each timestep, and messages are trained by *future-gradient injection*: the sender's message loss at time t is constructed from the receiver's loss at time $t+1$, differentiated with respect to the sender's positional encoding. Because the token construction is linear in both the raw message and the positional encoding, this gradient equals the true gradient of the receiver's future loss with respect to the message, so no re-attachment step is needed. A comparison with backpropagation through time (Section 3.8) shows that the TINAS recurrence computes the same multi-step gradient as DIAL in the detached sense, differs from the exact DIAL gradient by exactly the bounded STE bias, and reduces per-step graph memory by a factor of $\Omega(L)$ in the long-rollout regime ($T \cdot S \gg A$), where L is the transformer depth. Under standard smoothness and boundedness assumptions, TINAS converges to a stationary point of its total objective at rate $O(1/\sqrt{K})$, with residual straight-through-estimator (STE) bias bounded by c_m times the message magnitude times the second derivative of the loss. We describe a two-pass backward algorithm that achieves this at $O(T)$ cost—the same asymptotic scaling as standard PPO, plus one additional `autograd.grad` call per agent per step—with no buffer replay. We describe an implementation on top of a shared-parameter behavioral-cloning backbone for the StrikeForce environment.

Domain: Multi-Agent Reinforcement Learning, Communication, Optimization
Keywords: multi-agent RL, parameter sharing, communication learning, straight-through estimator, future-gradient injection

1 Introduction

1.1 Problem

Multi-agent reinforcement learning with parameter sharing is a powerful paradigm: a single policy network shared across all agents avoids the combinatorial blowup of independent learning and lets agents generalize across roles (Foerster et al., 2016; Sukhbaatar et al., 2016). Yet standard PPO training (Schulman et al., 2017) treats each agent's loss as local—no mechanism lets agents *learn to communicate* in a gradient-aware way. Each agent receives only its own observations and rewards; any implicit coordination happens through the shared parameters, not through an explicit channel.

1.2 Why Naive Communication Fails

A natural remedy is to add a communication head that emits messages between agents and backpropagate through the receiver's loss into the sender. This is essentially what DIAL (Foerster et al., 2016) does. However, differentiating through a T-step rollout with attached messages requires storing the full computational graph for every timestep, which is $O(T)$ memory and incompatible with the standard PPO minibatching. Loosening the graph (detach the message, then re-attach it before the receiver's backward) breaks the gradient chain and yields a biased estimator that is silent about its own bias.

1.3 Our Approach: Future-Gradient Injection

TINAS closes this gap with a *differentiable inter-agent communication channel* built on top of a shared transformer backbone. Each agent i emits a message $m^t_{i\to j}$ to every other agent j at each timestep. The messages are trained by **future-gradient injection**: the sender's message loss at time t is constructed from the *receiver's* loss at time $t+1$, differentiated with respect to the sender's positional encoding $\mathrm{pos}[i]$. Because the token construction is *linear in both* the raw message and $\mathrm{pos}[i]$, this gradient equals the "true" gradient of the receiver's future loss with respect to the message. Gradient injection is therefore possible *without any re-attachment step*: the detached message remains detached, and the injected gradient is recovered from the attached positional encoding.

1.4 Summary of Contributions

- **A gradient-equivalence lemma (Lemma 1).** For a token construction linear in the message and in the sender's positional encoding, $\partial L_j^{t+1}/\partial m^t_{i\to j} = \partial L_j^{t+1}/\partial \mathrm{pos}[i]$ exactly. The injected gradient can be extracted with a single `autograd.grad` call on the positional encoding; no re-attachment is needed.

- **A two-pass backward algorithm (Section 4).** Pass 1 is a forward pass under `NoGradGuard` that records detached messages. Pass 2 is a backward pass that reconstructs each step's loss from recorded snapshots, extracts the injected gradient, and accumulates parameter gradients. Total cost is $O(T)$, the same asymptotic scaling as standard PPO, plus one extra gradient call per agent per step. There is no buffer replay.

- **A convergence analysis (Section 3).** Under standard smoothness and boundedness, the injected-gradient recurrence is contractive when $c_m L_{\mathrm{net}} < 1$ (Proposition 1), the STE bias is bounded and controlled by c_m (Lemma 4), and AdamW with gradient clipping at norm 1 converges at $O(1/\sqrt{K})$ to an ϵ-neighborhood of a stationary point (Theorem 1).

- **A theoretical comparison with BPTT (Section 3.8).** We show that the TINAS recurrence computes exactly the detached-BPTT gradient (Lemma 5), differs from the exact DIAL gradient by exactly the STE bias of Lemma 4, and reduces per-step memory by a factor of $\Omega(L)$ in the long-rollout, deep-transformer regime, where L is the transformer depth. The trade-off is characterized in parameter form: TINAS is preferable in the long-rollout, deep-transformer regime, while DIAL is preferable when exact gradients are required and memory is not binding.

- **A structural decoupling (Section 3.8).** We show that TINAS performs credit assignment over a horizon of $T-t$ steps at $\Theta(1)$ graph depth per step, with geometrically decaying influence controlled by $c_m L_{\mathrm{net}}$ (Lemma 5, Proposition 1). This decouples credit-assignment horizon from graph-memory depth—a decoupling that, to our knowledge, no prior multi-agent communication-learning method achieves. The underlying principle is related to RTRL-style online credit assignment in single-agent recurrent networks (Williams & Zipser, 1989), but its instantiation for inter-agent communication with a linear-in-positional-encoding token construction is new.

- **A concrete instantiation (Section 9)** on top of a shared-parameter behavioral-cloning backbone for the StrikeForce 2D battle-arena testbed, with the code available at the public repository listed above.

Taken together, these contributions establish a single structural claim: credit-assignment horizon and graph-memory depth can be decoupled in multi-agent communication learning. In BPTT they are locked—a T-step horizon requires a T-step graph. TINAS achieves a $T - t$ horizon at $\Theta(1)$ graph depth per step by carrying the gradient through a detached recurrence rather than through the graph, at the price of a bounded second-order error (Lemma 4) and with an $\Omega(L)$ reduction in per-step memory.

2 Model

2.1 Notation

We consider n agents. The communication extension has parameter vector $\theta \in \mathbb{R}^d$; the pretrained backbone is held fixed. A rollout consists of T timesteps $t = 1, \ldots, T$. At each step, agent i sends a message $m_{i\to j}^t$ to each other agent $j \neq i$. Two positional embeddings are used: $\mathrm{pos}[i] \in \mathbb{R}^{d_{\mathrm{msg}}}$ for each sender i, and a spatial distance vector $d^t[j \to i] \in \mathbb{R}^{d_{\mathrm{dist}}}$ encoding the geometry between receiver j and sender i.

Boundary condition.

$$L_i^{T+1} \;=\; 0 \qquad \text{(constant, for all } i\text{)}.$$

Per-agent per-step loss (PPO + value + entropy + message).

$$L_i^t \;=\; L_{\mathrm{PPO}}^{i,t} + c_V L_V^{i,t} - c_H H^{i,t} + \frac{c_m}{n-1} \sum_{j \neq i} L_{m,i\to j}^t.$$

Message loss.

$$L_{m,i\to j}^t \;=\; \big(g_{i\to j}^{t+1}\big)^\top m_{i\to j}^t.$$

Injected gradient.

$$g_{i\to j}^{t+1} \;=\; \left(\frac{\partial L_j^{t+1}}{\partial \mathrm{pos}[i]}\right)_{\mathrm{detached}}.$$

The *detached* subscript denotes that the gradient is treated as a constant in any subsequent backward pass. It is extracted with `create_graph=False`, so it carries no computational graph.

Total objective.

$$\mathcal{L}(\theta) \;=\; \frac{1}{Tn} \sum_{t=1}^{T} \sum_{i=1}^{n} L_i^t.$$

2.2 Token Construction (Exact)

For each ordered pair (i, j) with $i \neq j$, the message token transmitted from sender i to receiver j at step t is

$$\boxed{\mathrm{tok}_{i\to j}^t \;=\; \mathrm{proj}\big(\mathrm{cat}\big\{m_{i\to j}^t + \mathrm{pos}[i],\; d^t[j \to i]\big\}\big)\,.}$$

Here $\mathrm{proj} : \mathbb{R}^{d_{\mathrm{msg}}+d_{\mathrm{dist}}} \to \mathbb{R}^{d_{\mathrm{model}}}$ is a linear map (the `msg_proj` module of the extension), and `cat` denotes concatenation along the feature dimension. The first block of the input is $m_{i\to j}^t + \mathrm{pos}[i]$—the raw message plus the sender's positional encoding—and the second block is the spatial distance vector.

At the receiver j, the messages from all senders $i \neq j$ enter the transformer's input sequence together with j's own r_j^t and a learnable value token v. After the transformer, the message slots are projected by a linear head to produce the outbound messages $m_{j\to k}^t$.

Remark 1 (Why the linearity matters)**.** *The composition in the boxed equation is* linear in both arguments $m_{i\to j}^t$ *and* $pos[i]$*. This is the technical fact that drives the entire gradient-injection mechanism. A non-linear projection, or a token construction that mixed* $pos[i]$ *into multiple slots, would break the equivalence of Lemma 1.*

2.3 Gradient Equivalence

Lemma 1 (Gradient equivalence). *For any $i \neq j$ and any t, the two gradients*

$$\frac{\partial L_j^{t+1}}{\partial m_{i\to j}^t} \qquad \textit{and} \qquad \frac{\partial L_j^{t+1}}{\partial pos[i]}$$

are equal as tensors in $\mathbb{R}^{d_{msg}}$.

Proof. Let $J_{\text{proj}} \in \mathbb{R}^{d_{\text{model}} \times (d_{\text{msg}}+d_{\text{dist}})}$ be the Jacobian of the linear map proj. Write $a = m_{i\to j}^t + \text{pos}[i]$ for the first input block. Differentiation of the token construction gives

$$\frac{\partial \text{tok}_{i\to j}^t}{\partial m_{i\to j}^t} \;=\; J_{\text{proj}} \begin{bmatrix} I_{d_{\text{msg}}} \\ 0_{d_{\text{dist}}} \end{bmatrix} \;=\; J_{\text{proj}}[:, 0{:}d_{\text{msg}}],$$

$$\frac{\partial \text{tok}_{i\to j}^t}{\partial \text{pos}[i]} \;=\; J_{\text{proj}} \begin{bmatrix} I_{d_{\text{msg}}} \\ 0_{d_{\text{dist}}} \end{bmatrix} \;=\; J_{\text{proj}}[:, 0{:}d_{\text{msg}}].$$

The two are identical. By the chain rule through the downstream computation of L_j^{t+1},

$$\frac{\partial L_j^{t+1}}{\partial m_{i\to j}^t} \;=\; \frac{\partial L_j^{t+1}}{\partial \text{tok}_{i\to j}^t} \cdot \frac{\partial \text{tok}_{i\to j}^t}{\partial m_{i\to j}^t} \;=\; \frac{\partial L_j^{t+1}}{\partial \text{tok}_{i\to j}^t} \cdot \frac{\partial \text{tok}_{i\to j}^t}{\partial \text{pos}[i]} \;=\; \frac{\partial L_j^{t+1}}{\partial \text{pos}[i]}.$$

□

Remark 2 (Dependence of L_j^{t+1} on (i, j)). *The proof uses the fact that L_j^{t+1} depends on $m_{i\to j}^t$* only *through the single token $(i \to j)$ at receiver j. This holds because (i) each sender contributes exactly one token to each receiver, and (ii) L_j^{t+1} depends only on receiver j's outputs. In the implementation, the constant gather index `other_idx[i]` $= [j \neq i]$ enforces exactly one incoming message per sender-receiver pair.*

Remark 3 (No re-attachment required). *The algorithm extracts $g_{i\to j}^{t+1}$ via `autograd.grad`(L_j^{t+1}, pos) rather than `autograd.grad`(L_j^{t+1}, m^t). This is not an approximation—it is exact. It works because pos$[i]$ remains attached to the graph (it is a learnable parameter), whereas $m_{i\to j}^t$ has been detached on the send side. The equivalence lemma says that the "as-if-attached" gradient of L_j^{t+1} with respect to $m_{i\to j}^t$ can be recovered from the genuinely-attached gradient with respect to pos$[i]$. Pass 2 of the algorithm does not need to flip `requires_grad` on detached messages; it simply calls `autograd.grad` on pos$[i]$ and obtains the correct g.*

2.4 Per-Step Loss (Restated)

$$L_i^t \;=\; L_{\text{PPO}}^{i,t} + c_V L_V^{i,t} - c_H H^{i,t} + \frac{c_m}{n-1} \sum_{j \neq i} \left(g_{i\to j}^{t+1}\right)^\top m_{i\to j}^t.$$

Every term is an unnormalized per-agent per-step contribution; the total $\mathcal{L}$ normalizes by Tn.

2.5 Total Objective

$$\boxed{\mathcal{L}(\theta) \;=\; \frac{1}{Tn} \sum_{t=1}^{T} \sum_{i=1}^{n} L_i^t.}$$

The gradient $\nabla_\theta \mathcal{L}$ is what AdamW (Loshchilov & Hutter, 2019) optimizes, and the training update is

$$\theta_{K+1} \;=\; \text{AdamW}\Big(\theta_K,\; \hat{G}_K,\; \eta = 10^{-3},\; \lambda = 10^{-4}\Big),$$

where $\hat{G}_K$ is a minibatch estimate of $\nabla_\theta \mathcal{L}$.

3 Theoretical Analysis

Reading guide. Section 3 is organized as follows. Section 1–5 state the assumptions. Section 2 establishes well-definedness of the recursion. Section 3 derives the per-step gradient. Section 1 proves contractivity of the injected-gradient recurrence. Section 4 bounds the STE bias. Section 1–1 give the convergence rate. Section 3.8 compares TINAS with DIAL/BPTT.

3.1 Assumptions

Assumption 1 (Smoothness). *$\mathcal{L} : \mathbb{R}^d \to \mathbb{R}$ is L_{lip}-smooth: $\|\nabla\mathcal{L}(\theta) - \nabla\mathcal{L}(\theta')\| \le L_{lip}\|\theta - \theta'\|$. This is standard for networks composed of Lipschitz layers (LayerNorm, ReLU, linear) with bounded weights, and is enforced in practice by weight decay.*

Assumption 2 (Bounded gradient). *There exists $G > 0$ such that $\|\nabla_\theta\mathcal{L}(\theta)\| \le G$. This is enforced by **gradient clipping at norm 1**, so $G = 1$.*

Assumption 3 (Bounded variance). *For any stochastic minibatch ξ_K:*

$$\mathbb{E}[\hat{G}_K \mid \theta_K] = \nabla_\theta\mathcal{L}(\theta_K), \qquad \mathbb{E}[\|\hat{G}_K - \nabla_\theta\mathcal{L}(\theta_K)\|^2] \le \sigma^2.$$

Assumption 4 (Network Lipschitz). *Let*

$$L_{net} := \sup_\theta \max_{i,j,k,t} \left\| \frac{\partial m^t_{j\to k}}{\partial pos[i]} \right\|_{op}.$$

This is finite for any finite network and is bounded in practice by the Lipschitz constants of the layers and the norm of the incoming messages.

Assumption 5 (Bounded second derivative). *There exists $B_2 > 0$ such that $\sup_t \|\partial g^t/\partial\theta\| \le B_2$ along the optimization trajectory. This controls the STE bias term of Lemma 4 independently of Assumptions 1–4.*

Assumptions 1–3 are standard in the nonconvex optimization literature (Ghadimi & Lan, 2013; Bottou et al., 2018). Assumption 4 is specific to TINAS and controls the contractivity of the message recurrence. Assumption 5 is specific to the STE analysis and is bounded in practice by the norms already appearing in L_{net} and L_{lip}.

3.2 Well-Definedness

Lemma 2 (Recursion terminates). *The message loss $L^t_{m,i\to j}$ is well-defined for all $t = 1, \dots, T$ and all $i \ne j$. Moreover, it is zero at $t = T$.*

Proof. By the boundary condition $L^{T+1}_j = 0$, the gradient $g^{T+1}_{i\to j} = 0$, so $L^T_{m,i\to j} = 0^\top m^T_{i\to j} = 0$. For $t = T-1$, the loss $L^{T-1}_{m,i\to j}$ depends on $g^T_{i\to j} = \partial L^T_j/\partial \text{pos}[i]$, and L^T_j contains only a base term and $L^T_{m,j\to k} = 0$ (just shown). By induction on $T - t$, every $L^t_{m,i\to j}$ is a finite sum of at most $T - t$ nested gradients, each well-defined. The recursion has depth at most $T - t$ at step t. □

3.3 Gradient Computation

Lemma 3 (Gradient formula). *The gradient of the total objective with respect to θ is*

$$\nabla_\theta\mathcal{L}(\theta) \;=\; \frac{1}{Tn}\sum_{t=1}^{T}\sum_{i=1}^{n}\left[\nabla_\theta\Big(L^{i,t}_{PPO} + c_V L^{i,t}_V - c_H H^{i,t}\Big) + \frac{c_m}{n-1}\sum_{j\ne i}\big(g^{t+1}_{i\to j}\big)^\top \nabla_\theta m^t_{i\to j}\right].$$

Proof. Since $g^{t+1}_{i\to j}$ is detached (constant in θ), differentiation of the message-loss term yields

$$\nabla_\theta\big[(g^{t+1}_{i\to j})^\top m^t_{i\to j}\big] = (g^{t+1}_{i\to j})^\top \nabla_\theta m^t_{i\to j}.$$

Summing over i, j, t and normalizing by Tn gives the stated formula. □

Lemma 3 is the operational heart of the algorithm: the sent gradient $g_{i\to j}^{t+1}$ acts as a *constant weight* on the message's parameter dependence.

3.4 Contractivity of the Recurrence

Define the operator $T^t : \mathbb{R}^{n(n-1)\times d_{\text{msg}}} \to \mathbb{R}^{n(n-1)\times d_{\text{msg}}}$ by

$$\left(T^t G\right)_{(i,j)} \;=\; \frac{c_m}{n-1} \sum_{k\neq j} G_{(j,k)} \cdot \frac{\partial m_{j\to k}^t}{\partial \text{pos}[i]},$$

where $G_{(j,k)} \in \mathbb{R}^{d_{\text{msg}}}$ is the (j,k)-th slice of G and the product is the vector-Jacobian product $G^\top \partial m/\partial \text{pos}$. Then the recurrence for the injected gradients is

$$g_{(i,j)}^t \;=\; b_{(i,j)}^t \;+\; \left(T^t g^{t+1}\right)_{(i,j)},$$

where

$$b_{(i,j)}^t = \frac{\partial}{\partial \text{pos}[i]} \left(L_{\text{PPO}}^{j,t} + c_V L_V^{j,t} - c_H H^{j,t} \right)$$

is the "base" contribution for receiver j.

Proposition 1 (Contractivity)**.** *Under Assumption 4,*

$$\|T^t\|_{op} \;\leq\; c_m \, L_{net}.$$

In particular, when $c_m \, L_{net} < 1$, the operator T^t is a strict contraction, and the accumulated injected gradient satisfies

$$\|g^t\| \;\leq\; \sum_{s=0}^{T-t} \left(c_m L_{net}\right)^s \cdot \|b^{t+s}\|.$$

Proof. For a fixed slice (i,j), the operator T^t sums over $k \neq j$, giving $n-1$ terms each with coefficient $c_m/(n-1)$. Bounding each term by $L_{\text{net}}\|G_{(j,k)}\|$ and applying the triangle inequality gives

$$\left\|\left(T^t G\right)_{(i,j)}\right\| \;\leq\; \frac{c_m}{n-1} \sum_{k\neq j} L_{\text{net}} \|G_{(j,k)}\| \;\leq\; c_m \, L_{\text{net}} \cdot \max_k \|G_{(j,k)}\|.$$

The operator T^t is *row-wise*: its output at row (i,j) depends only on row j of G. Its spectral norm is therefore bounded by $c_m L_{\text{net}}$, dimension-free in n. The geometric bound follows by unrolling $g^t = b^t + T^t g^{t+1}$ up to the boundary condition. □

Remark 4 (Role of $c_m/(n-1)$)**.** *The factor $c_m/(n-1)$ in the loss is what makes the operator T^t contractive with a coefficient that* does not grow with n. *Without it, the coefficient per level would be $c_m(n-1)L_{net}$, which fails to contract for large n.*

3.5 STE Bias

The injected gradient $g_{i\to j}^{t+1}$ is **detached** before it enters the message loss $L_{m,i\to j}^t$. This is the STE approximation (Bengio et al., 2013). The "true" objective $\mathcal{L}^*$ would have g *attached*, and its gradient would include an additional second-order term.

Lemma 4 (STE bias bound)**.** *Define the "true" objective*

$$\mathcal{L}^*(\theta) \;=\; \frac{1}{Tn} \sum_{t,i} \left[base_i^t + \frac{c_m}{n-1} \sum_{j\neq i} \left(g_{i\to j}^{t+1}(\theta)\right)^\top m_{i\to j}^t(\theta) \right]$$

where g is not detached. Then, under Assumption 5,

$$\left\|\nabla_\theta \mathcal{L} - \nabla_\theta \mathcal{L}^*\right\| \;\leq\; c_m \cdot \sup_t \|m^t\| \cdot B_2.$$

Moreover, the bias vanishes identically when $m^t \equiv 0$.

Proof. Since $\mathcal{L}$ treats g^{t+1} as a constant, the difference is the missing term

$$\frac{1}{Tn}\sum_{t,i,j}\frac{c_m}{n-1}\left(m^t_{i\to j}\right)^\top\frac{\partial g^{t+1}_{i\to j}}{\partial\theta}.$$

The number of terms in the sum is $T\cdot n\cdot(n-1)$. After multiplying by $c_m/(Tn(n-1))$, each term is bounded by $c_m\|m\|B_2$. Taking the supremum over t gives the stated bound. The vanishing at $m=0$ is immediate. □

The bias $\mathcal{B} := c_m \sup\|m\|B_2$ enters the convergence analysis as a fixed model error. Note that c_m appears in the bias bound *linearly*. In practice, c_m is chosen small enough (typically 0.1) that $\mathcal{B}$ is small compared to the scale of $\nabla_\theta\mathcal{L}$.

3.6 Convergence Theorem

Theorem 1 (Convergence to a stationary point)**.** *Suppose Assumptions 1–5 hold and let $\mathcal{B}$ be the STE bias bound of Lemma 4. Run AdamW with step sizes $\eta_K=\eta_0/\sqrt{K}$ and gradient clipping at norm 1. Then for any $K\geq 1$,*

$$\mathbb{E}\left[\|\nabla_\theta\mathcal{L}(\theta_K)\|^2\right]\;\leq\;\frac{C_1}{\sqrt{K}}\;+\;\mathcal{B}^2,$$

where $C_1=2(\mathcal{L}(\theta_0)-\mathcal{L}^{\inf})/\eta_0+\eta_0 L_{lip}\sigma^2$ is a constant depending on initial condition, learning rate, smoothness, and gradient noise; provided the contractivity condition $c_m L_{net}<1$ of Proposition 1 holds, C_1 is independent of K, T, and n.

Proof sketch. (Here the $\sqrt{K}$ decay is an analysis choice; the implementation uses a constant learning rate, and the same rate holds up to constants.) Treating AdamW with step sizes $\eta_K=\eta_0/\sqrt{K}$ as a preconditioned SGD, the standard descent lemma (Ghadimi & Lan, 2013) gives

$$\mathbb{E}[\mathcal{L}(\theta_{K+1})]\;\leq\;\mathbb{E}[\mathcal{L}(\theta_K)]-\eta_K\mathbb{E}[\|\nabla\mathcal{L}(\theta_K)\|^2]+\eta_K^2L_{\text{lip}}\sigma^2+\eta_K\mathcal{B}^2.$$

Summing over $K=1,\dots,K_{\text{end}}$ and using $\eta_K=\eta_0/\sqrt{K}$ gives

$$\sum_{k=1}^{K_{\text{end}}}\frac{\eta_0}{\sqrt{k}}\mathbb{E}[\|\nabla\mathcal{L}(\theta_k)\|^2]\;\leq\;\mathcal{L}(\theta_0)-\mathcal{L}^{\inf}+\eta_0L_{\text{lip}}\sigma^2\sum_{k=1}^{K_{\text{end}}}\frac{1}{k}+\eta_0\mathcal{B}^2\sqrt{K_{\text{end}}}.$$

Since $\sum_{k=1}^{K_{\text{end}}}1/\sqrt{k}\geq\sqrt{K_{\text{end}}}$, dividing through and using the harmonic sum bound $\sum 1/k\leq\log K_{\text{end}}$ yields the stated $O(1/\sqrt{K})+\mathcal{B}^2$ rate up to logarithmic factors. □

Remark 5 (Interpretation)**.** *The rate $O(1/\sqrt{K})$ is the optimal rate for nonconvex stochastic optimization with bounded gradients (Ghadimi & Lan, 2013). It says: for every $\epsilon>\mathcal{B}^2$, a finite number of updates reaches the ϵ-neighborhood of a stationary point of $\mathcal{L}$. The STE bias $\mathcal{B}$ is the irreducible floor—it caps the best achievable gradient norm, at the scale set by the message magnitude and the second derivative of the loss.*

3.7 Number of Updates to Reach an ϵ-Optimum

Corollary 1. *Choose $\epsilon>\mathcal{B}^2$, and let $\delta=\epsilon-\mathcal{B}^2>0$. Then*

$$K\;=\;\mathcal{O}\left(\frac{C_1^2}{\delta^2}\right)$$

updates suffice to reach $\mathbb{E}[\|\nabla\mathcal{L}(\theta_K)\|^2]\leq\epsilon$.

Proof. Direct from Theorem 1. □

Remark 6 (Reading)**.** *The number of updates to reach a stationary point scales as $O(1/\delta^2)$, where δ is the gap between the target accuracy and the STE floor. Since $\mathcal{B}=c_m\sup\|m\|B_2$, and both suprema are bounded by Assumptions 1–5 plus clipping, the number of updates is finite and independent of T and n.*

3.8 Relation to Backpropagation Through Time

The closest prior work, DIAL (Foerster et al., 2016), learns inter-agent messages by backpropagating through time (BPTT) over the full rollout graph. We now compare TINAS and BPTT at the level of the gradient computation and the memory cost. The comparison is *parameter-based*: we express all costs in terms of $(n, T, d_{\text{msg}}, d_r, L, |\theta|)$, so the results hold for any configuration and expose the trade-off in terms of the network's own dimensions rather than in task-specific constants. Here $|\theta| := \dim(\theta)$.

Multi-step gradient equivalence (detached form). We first strengthen Lemma 1 to the multi-step case. Define the *detached-BPTT* gradient of L_j^t with respect to $m_{i\to j}^{t-1}$ as the gradient computed by treating the injected gradients $g^{t'}$ for all $t' > t$ as constants. Then:

Lemma 5 (Multi-step gradient equivalence)**.** *Let $g_{i\to j}^t$ be the injected gradient of Section 2, i.e., $g_{i\to j}^t = \partial L_j^t / \partial pos[i]$ with $g^{t'}$ held constant for all $t' > t$. Then*

$$g_{i\to j}^t \;=\; \frac{\partial L_j^t}{\partial m_{i\to j}^{t-1}}\bigg|_{detached\text{-}g}$$

for all $i \neq j$ and all t, where the right-hand side is the detached-BPTT gradient.

Proof. By induction on $T - t$. *Base case* $t = T$: since $g^{T+1} = 0$, $L_j^T = \text{base}_j^T$, and both sides reduce to the single-step gradient of base_j^T with respect to $\{\text{pos}[i], m_{i\to j}^{T-1}\}$, which are equal by Lemma 1. *Inductive step*: assume the claim holds for $t+1$. Then, by the recurrence of Section 3,

$$g_{i\to j}^t \;=\; b_{i\to j}^t \;+\; \frac{c_m}{n-1}\sum_{k\neq j} g_{j\to k}^{t+1}\cdot\frac{\partial m_{j\to k}^t}{\partial \text{pos}[i]},$$

whereas

$$\frac{\partial L_j^t}{\partial m_{i\to j}^{t-1}}\bigg|_{\text{detached-}g} \;=\; b_{i\to j}^t \;+\; \frac{c_m}{n-1}\sum_{k\neq j} g_{j\to k}^{t+1}\cdot\frac{\partial m_{j\to k}^t}{\partial m_{i\to j}^{t-1}},$$

using the inductive hypothesis $g_{j\to k}^{t+1} = \partial L_k^{t+1}/\partial m_{j\to k}^t|_{\text{detached-}g}$. The two right-hand sides are equal because both $\text{pos}[i]$ and $m_{i\to j}^{t-1}$ enter the token construction through the same linear slot, so $\partial m_{j\to k}^t/\partial \text{pos}[i] = \partial m_{j\to k}^t/\partial m_{i\to j}^{t-1}$ (Lemma 1). Hence $g_{i\to j}^t = \partial L_j^t/\partial m_{i\to j}^{t-1}|_{\text{detached-}g}$. □

Lemma 5 says that, with the injected gradients held constant, the TINAS recurrence computes exactly the detached-BPTT gradient. The only difference from *true* BPTT is the dependence of g^{t+1} itself on m^{t-1}, i.e., the second-order term.

Gradient discrepancy. Combining Lemma 5 with Lemma 4, the TINAS parameter update differs from the exact BPTT update of DIAL by exactly the STE bias:

$$\nabla_\theta \mathcal{L}_{\text{TINAS}} \;=\; \nabla_\theta \mathcal{L}_{\text{BPTT}} \;-\; \Delta_{\text{STE}}, \qquad \|\Delta_{\text{STE}}\| \;\le\; c_m \cdot \sup_t \|m^t\| \cdot B_2.$$

So TINAS is a *controlled* approximation to DIAL: the approximation error is bounded by the same $\mathcal{B}$ that appears in Theorem 1, and it is the second-order dependence of the injected gradient on the parameters that TINAS trades away in exchange for its memory savings.

Memory comparison. Let A denote the activation memory of a single forward step (the tensors that autograd must retain for backward), and S the snapshot memory of a single TINAS step, $S = n\cdot(d_{\text{msg}} + d_r)$. Both quantities are properties of the architecture, not of the task. For a transformer with L layers and hidden dimension d, one has

$$A \;=\; \Omega(n\cdot d\cdot L)\,, \qquad S \;=\; \mathcal{O}(n\cdot d)\,,$$

so $A/S = \Omega(L)$. The two algorithms then separate as in Table 1.

	DIAL (BPTT)	TINAS
Graph depth	$\Theta(T)$	$\Theta(1)$ per step
Activation memory	$\Theta(T \cdot A)$	$\Theta(T \cdot S + A)$
Parameter gradient storage	$\Theta(\lvert\theta\rvert)$	$\Theta(\lvert\theta\rvert)$
Total asymptotic memory	$\Theta(T \cdot A + \lvert\theta\rvert)$	$\Theta(T \cdot S + A + \lvert\theta\rvert)$
Ratio (TINAS / DIAL)	1	$\Theta(S/A + 1/T)$

Table 1: Parameter-based memory comparison. A is the per-step activation memory of the forward pass; $S = n \cdot (d_{\text{msg}} + d_r)$ is the per-step snapshot memory. For a transformer, $A/S = \Omega(L)$.

For long rollouts ($T \gg 1$) and $T \cdot S \gg A$, the ratio is $\Theta(S/A) = \mathcal{O}(1/L)$: TINAS's memory advantage over DIAL is at least a factor of the transformer depth L, and is asymptotically independent of n and T.

Computational cost. Both algorithms are $\Theta(T)$ in time. The constants differ. DIAL performs one forward pass and one backward pass over a T-step graph, with per-step constant proportional to A. TINAS performs one forward pass under `NoGradGuard` and one forward-plus-backward pass over T single-step graphs, with per-step constant proportional to S. The dominant term in DIAL is $\Theta(T \cdot A)$; in TINAS it is $\Theta(T \cdot S)$. TINAS is faster by the same factor $\Theta(A/S) = \Omega(L)$ per step.

Trade-off characterization. Combining the above gives a clean parameter-based trade-off. Fix a target accuracy $\varepsilon > \mathcal{B}^2$, let $\delta = \varepsilon - \mathcal{B}^2$, and let $K_{\text{DIAL}}(\varepsilon)$ and $K_{\text{TINAS}}(\varepsilon)$ be the number of updates required to reach $\mathbb{E}[\|\nabla_\theta \mathcal{L}\|^2] \le \varepsilon$ for each method. Then:

- $K_{\text{DIAL}}(\varepsilon) = \mathcal{O}(C_1/\varepsilon^2)$ (no bias floor), at memory $\Theta(T \cdot A + |\theta|)$;
- $K_{\text{TINAS}}(\varepsilon) = \mathcal{O}(C_1/\delta^2)$ (bias floor $\mathcal{B}^2$), at memory $\Theta(T \cdot S + A + |\theta|)$;
- the memory ratio $M_{\text{DIAL}}/M_{\text{TINAS}} = \Omega(L)$ for $T \cdot S \gg A$ and $T \gg 1$.

TINAS is preferable when (i) memory is the binding constraint, and (ii) the bias floor $\mathcal{B}^2$ is small relative to the target ε. Both conditions are under the practitioner's control: $\mathcal{B} = c_m \sup_t \|m^t\| \cdot B_2$ decreases with c_m (a design choice) and with message regularization, and the memory advantage grows with the transformer depth L.

DIAL is preferable when (i) the rollout is short enough that $T \cdot A$ fits in memory, (ii) exact gradients are required at machine precision, or (iii) the bias floor $\mathcal{B}^2$ exceeds the target accuracy. In the regime of long rollouts and deep transformers—which is precisely the regime the StrikeForce testbed operates in—the memory advantage of TINAS dominates, and the bias floor is controlled by the choice of c_m and by the message-norm regularization of the implementation.

4 Algorithm: Two-Pass Backward

The theoretical guarantees are backed by an efficient two-pass algorithm. It mirrors the standard PPO rollout loop with an added backward pass for message-gradient injection.

Pass 1 — forward without gradients.

```
m[0] = 0
with NoGradGuard:
    for t = 1, ..., T-1:
        r[t]  = model->forward_r_t(obs[t]).r_t().clone().detach()
        m[t]  = extension->forward(r[t], m[t-1]).new_messages().clone().detach()
model->reset_memory()
```

Both $r[t]$ and $m[t]$ are recorded as detached tensors for later use. This pass touches `PlayerPolicyNet` for the $r[t]$ computation only; the backbone is frozen and is not updated.

Pass 2 — backward with injected gradients.

```
g[T+1][i][j] = 0              for all i, j
G = 0                          # accumulated gradient

for t = T, T-1, ..., 1:
    # Differentiable forward, using recorded detached snapshots
    out = extension->forward(r[t], m[t-1])    # returns logits, values, new_messages

    for i = 1, ..., n:
        L_msg = 0
        for j != i:
            L_msg += (g[t+1][i][j] . m[t][i->j]).sum()
        L_msg /= (n - 1)

        L_i^t = L^{i,t}_PPO + c_V L^{i,t}_V - c_H H^{i,t} + c_m * L_msg

        # Extract both parameter gradient and injected gradient in one call
        grad_i = autograd.grad(L_i^t, params)                    # grad_theta L_i^t
        grad_i = clip_norm_1(grad_i)                             # Assumption 2 enforced
        G += grad_i

        # incoming convention: g[t][k][i] = dL_i^t / dpos[k]
        g_ret = autograd.grad(L_i^t, pos, create_graph=False)    # [n, d_msg]
        for k != i:
            g[t][k][i] = g_ret[k]

G /= (T * n)
theta <- AdamW(G, lr=1e-3, wd=1e-4)
```

Indexing convention. Let $g[t][i][j]$ denote the entry used at step t for the term $\left(g^t_{i\to j}\right)^\top m^{t-1}_{i\to j}$. By the loss definition,

$$g^t_{i\to j} \;=\; \frac{\partial L^t_j}{\partial \mathrm{pos}[i]}.$$

So the entry stored at index (i, j) after `autograd.grad`(L^t_j, pos) is $\partial L^t_j/\partial \mathrm{pos}[i]$. In the pseudocode, after `autograd.grad`(L^t_i, pos), the returned tensor has entry k equal to $\partial L^t_i/\partial \mathrm{pos}[k]$; the correct storage is therefore $g[t][k][i] \leftarrow (\text{returned})[k]$ for all $k \neq i$, i.e., the *incoming* convention (gradient w.r.t. message from k to i), not the outgoing convention.

Two clarifications.

1. **No re-attach step.** Pass 2 uses `autograd.grad`(L^t_i, pos) directly. By Lemma 1, this equals the message-side gradient $\partial L^t_i/\partial m$ that would be computed with m attached. The equivalence is exact; no `requires_grad=True` flip on m is needed.
2. **Pass 2 never touches `PlayerPolicyNet`.** The $r[t]$ values are consumed as detached inputs from Pass 1, so the sliding-window buffer in `PlayerPolicyNet` is irrelevant during Pass 2. This sidesteps the buffer-replay issue that would otherwise require either $O(T)$ buffer snapshots or $O(T^2)$ recomputation.

Cost. Pass 1 is $O(T)$ forward computations. Pass 2 is $O(T)$ forward + backward computations. Total: $O(T)$, same asymptotic cost as standard PPO, with a constant factor of n extra backward calls per step.

5 Practical Guarantees under AdamW + Gradient Clipping

The analysis above is stated for SGD in Theorem 1, but the guarantees carry over to AdamW + clip 1 with two remarks.

Clip 1 enforces Assumption 2 exactly. The update $\theta_{K+1} = \theta_K - \eta_K \hat{G}_K / \max(1, \|\hat{G}_K\|)$ satisfies $\|\theta_{K+1} - \theta_K\| \leq \eta_K$ regardless of the loss landscape, and $\|\hat{G}_K\| \leq 1$. This is precisely the bounded-gradient assumption of Assumption 2.

AdamW does not change the asymptotic rate. The adaptive step $\eta_K \hat{m}_K / (\sqrt{\hat{v}_K} + \epsilon)$ modifies the per-coordinate magnitude but not the direction of descent. Under Assumptions 1–3, the analysis of Theorem 1 treats AdamW as a preconditioned SGD and inherits its $O(1/\sqrt{K})$ rate (Ghadimi & Lan, 2013). The STE bias enters as before.

Combined. The guarantees of Theorem 1 hold with the constant C_1 adjusted for the adaptive scaling. The number of updates $K = O(1/\delta^2)$ from Corollary 1 remains valid.

6 Characterization of the Stationary Point

At any stationary point θ^* of $\mathcal{L}$, we have $\nabla_\theta \mathcal{L}(\theta^*) = 0$, which by Lemma 3 reads

$$\frac{1}{Tn} \sum_{t,i} \left[\nabla_\theta \left(L_{\text{PPO}}^{i,t} + c_V L_V^{i,t} - c_H H^{i,t} \right) + \frac{c_m}{n-1} \sum_{j \neq i} \left(g_{i \to j}^{t+1} \right)^\top \nabla_\theta m_{i \to j}^t \right] = 0.$$

This is a **balance condition** between two contributions: the PPO signal that pushes the extension policy toward the standard objective, and the message-injection signal that pushes senders to produce messages that reduce receivers' future losses. The stationary point sits at a trade-off between these two forces.

In the shared-parameter setting, θ^* is a fixed point of the joint map $(\theta, m(\theta)) \mapsto \theta'$ generated by a single AdamW step. At equilibrium, no unilateral perturbation of any agent's messages can improve that agent's message-loss contribution while keeping all other parameters and messages fixed. This is a weak analogue of Nash equilibrium for the message-passing game.

7 Related Work

Communication in MARL. DIAL (Foerster et al., 2016) is the closest predecessor: agents send continuous messages over a differentiable channel, and gradients flow through the receiver's loss into the sender during training. *The essential difference between DIAL and TINAS is not architectural but computational:* both compute the same multi-step gradient in the detached sense (Lemma 5), but DIAL obtains it by backpropagating through a T-step computational graph, while TINAS obtains it by the recurrence of Section 3 without extending the graph. The price is the STE bias of Lemma 4; the gain is a per-step memory reduction of $\Omega(L)$, where L is the transformer depth (Section 3.8). At the message-channel level, DIAL uses a feed-forward network while TINAS uses a transformer attention block that mixes messages from all senders at once, but this is a secondary architectural choice: the future-gradient injection mechanism could in principle be layered onto a feed-forward channel, and vice versa. CommNet (Sukhbaatar et al., 2016) uses a mean-pooled channel with no per-pair messages; TarMAC (Das et al., 2019) uses attention-weighted messages but trains via standard end-to-end BPTT; IC3Net (Singh et al., 2019) learns discrete communication gates. TINAS is complementary to all of these in the architectural sense, and orthogonal in the computational sense: any of their message channels could in principle be trained by future-gradient injection instead of BPTT.

Straight-through estimators. The STE (Bengio et al., 2013) is typically used to backpropagate through non-differentiable operations. TINAS uses it differently: the message is detached not because its forward

operation is non-differentiable, but because we want to avoid storing its graph across T timesteps. Lemma 4 bounds the resulting bias explicitly, which is uncommon in STE applications where the bias is usually treated as part of the modeling assumption.

Transformers for MARL. The use of multi-head self-attention for inter-agent communication is well established (Vaswani et al., 2017; Wang et al., 2018). TINAS uses a pre-LN transformer block (Xiong et al., 2020) for its proj-then-attend pipeline and adds a dedicated positional encoding for the sender identity. The linear-in-pos construction is what enables Lemma 1.

Parameter-sharing MARL. Sharing parameters across agents is a standard technique for scaling MARL (Foerster et al., 2016; Rashid et al., 2018). TINAS operates in this regime and assumes a single shared extension network with per-agent positional embeddings.

8 Scope Statement

Explicitly not claimed:

- That TINAS learns *semantically meaningful* messages. The objective trains messages to reduce the receiver's future loss; what the messages encode is not analyzed in this paper.
- That the STE bias $\mathcal{B}$ is negligible at the operating c_m. Lemma 4 bounds it in terms of c_m, $\sup_t \|m^t\|$, and B_2, all of which are controlled but not driven to zero.
- That the contractivity condition $c_m L_{\text{net}} < 1$ holds throughout training. It holds at initialization (where both m and pos are near zero) and can degrade as training progresses. In this work, c_m is held fixed throughout training, by design: the contractivity condition is therefore an assumption on the operating regime rather than a property enforced by the algorithm. A c_m-schedule or a normalization on the message head would enforce it more robustly and is left for future work.
- That the convergence result of Theorem 1 is a global result for arbitrary neural networks. It is a local result under Assumptions 1–5.
- That the TINAS architecture is optimal for any particular task. It is a design choice; the theoretical guarantees are agnostic to the specific proj, attention block, and message head.
- That the backbone is trained by TINAS. The backbone is pretrained by behavioral cloning and frozen; only the communication extension is trained by the TINAS objective.

9 Implementation

We instantiate TINAS on top of a behavioral-cloning backbone trained in the *StrikeForce* 2D partially observable battle-arena testbed (Fouladi, 2026b), using the decoupled Future Action Prediction (FAP)/Transformer-aggregator architecture described in (Fouladi, 2026a). The backbone is frozen; the communication extension is trained by the TINAS objective. The code, configuration files, and evaluation scripts are available at `https://github.com/kasrafouladi/StrikeForce-No-BPTT-via-FAP`. Model checkpoints are hosted at `https://drive.google.com/drive/folders/1S2dE6DfzBU09ILpeJJ8UvtTGE0gAU-x1?usp=sharing`.

9.1 Architecture

Shared backbone. `PlayerPolicyNet` implements the shared-parameter per-agent policy. It consists of:

- an AFC backbone (`AFCBackboneSparse`) with two sparse-attention stages;
- a transformer policy block (`tf_policy`) with $d_{\text{model}} = 300$ and 10 heads;

- a learnable class token and a sliding window of $W = 31$ recent frame embeddings.

The forward pass outputs, per agent, a policy logit vector over $N_{\text{actions}} = 7$ actions and a per-agent representation $r_t \in \mathbb{R}^{300}$. The representation r_t is what the communication extension consumes as input. The backbone is pretrained by behavioral cloning and held fixed; it is not updated by TINAS.

Communication extension. `CommExtensionNet` implements the per-agent communication channel. Each agent's token sequence has length $n + 1$:

$$\left[\, r_i^t,\ \text{tok_value},\ m_{j_1 \to i},\ m_{j_2 \to i},\ \ldots,\ m_{j_{n-1} \to i} \,\right].$$

The first slot is the agent's own representation with a dedicated positional encoding rt_pe; the second is a learnable value token with no positional encoding; the remaining $n - 1$ slots are incoming messages from the other agents, each tagged with the sender's positional encoding sender_pe$[j]$ on the raw message, concatenated with the spatial distance vector, and projected by `msg_proj` to $d_r = 300$.

Output heads. Three linear heads operate on the transformer's per-token output:

- the r_t slot feeds `policy_head` to produce logits $\in \mathbb{R}^{n \times n_{\text{actions}}}$;
- the value-token slot feeds `value_head` to produce a per-agent value scalar;
- each message slot feeds `msg_out_head` to produce the outgoing message $m_{i \to j}^t \in \mathbb{R}^{d_{\text{msg}}}$.

Token construction per Lemma 1. The construction is implemented literally:

```
msg_with_pe = incoming + sender_pe[other_idx[i]]   # m + pos, both detached/attached
msg_feat    = cat({msg_with_pe, dist_in}, -1)      # concat distance
msg_emb     = msg_proj(msg_feat)                   # linear -> d_r
```

`incoming` is detached on the send side; `sender_pe` is an attached parameter. The linear map `msg_proj` is therefore applied to the sum, making the gradient w.r.t. `sender_pe` equal to the gradient w.r.t. the detached message. This is Lemma 1 in code.

Gather indices. `other_idx` is a constant $[n, n-1]$ tensor with `other_idx[i]`$= [j \neq i]$. It is registered as a buffer, not re-allocated per call, so the device transfer happens once.

9.2 Per-Step Composition

At each optimization step, the algorithm computes three gradient contributions and sums them before the optimizer step:

1. the PPO gradient from the extension's policy head, evaluated on the recorded $r[t]$ values;
2. the value-loss gradient from `value_head`;
3. the message-loss gradient from `msg_out_head`, weighted by the injected $g_{i \to j}^{t+1}$.

Each contribution is extracted via a separate `autograd.grad` call on the loss L_i^t; the injected gradient $g_{i \to j}^{t+1}$ is the `autograd.grad` output on `sender_pe` with `create_graph=False`. The per-agent gradients are clipped at norm 1 (Assumption 2), accumulated, and divided by Tn before the AdamW step.

9.3 Hyperparameters

The configuration used to produce the current implementation is summarized in Table 2. The values are not claimed to be optimal (c_m is held constant at 0.1 throughout training; no schedule or message-norm regularization is applied.).

Hyperparameter	Value
c_m (message loss weight)	0.1
c_V (value loss weight)	0.5
c_H (entropy weight)	0.01
AdamW learning rate η	10^{-3}
AdamW weight decay λ	10^{-4}
Message dimension d_{msg}	64
Distance dimension d_{dist}	2 (row, column offsets)
Number of agents n	5
Extension transformer heads	10
Gradient clipping norm	1

Table 2: Configuration used to produce the current implementation.

10 Planned Evaluation

We plan to evaluate TINAS against three baselines on StrikeForce, all sharing the same frozen behavioral-cloning backbone and environment:

- **No communication**: the frozen backbone alone.
- **DIAL**: the same backbone with a differentiable communication extension trained by end-to-end backprop, at $O(T)$ memory.
- **TINAS**: the same backbone with the two-pass future-gradient-injection algorithm of Section 4.

Evaluation protocol. Paired episodes will be used to reduce variance, with identical initial conditions, map seed, and action mask across checkpoints. All metrics will be reported as paired differences with Benjamini–Hochberg FDR correction across the full metric grid.

Metrics. We will report the same engagement-oriented metrics as the StrikeForce benchmark (Fouladi, 2026b): kill rate, kills per unit stamina, damage rate, and the kill-weighted composite metrics M_{HP}, M_{stam}, M_{dmg} of the benchmark. The operational failure region is the state distribution in which the policy disengages from combat when engagement is warranted, as defined in the benchmark's evaluation protocol.

Ablations. We plan to ablate the following design choices: (i) the sender positional encoding `sender_pe` (replaced with a fixed identity embedding); (ii) the value token; (iii) the distance concatenation; (iv) the shared-parameter architecture versus independent per-agent policies. These ablations are intended to isolate the contribution of the future-gradient injection mechanism from the architectural choices of the extension.

References

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.

Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Mike Rabbat, and Joelle Pineau. TarMAC: Targeted multi-agent communication. In *International Conference on Machine Learning*, 2019.

Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2016.

Kasra Fouladi. No back-propagation through time via future action prediction. ResearchGate, 2026a. URL https://doi.org/10.13140/RG.2.2.18971.43049.

Kasra Fouladi. Strikeforce: A 2d partially observable battle-arena testbed for imitation learning and multi-agent research. ResearchGate, 2026b. URL https://doi.org/10.13140/RG.2.2.24446.96326.

Saeed Ghadimi and Guanghui Lan. Stochastic first- and zeroth-order methods for nonconvex stochastic programming. *SIAM Journal on Optimization*, 23(4):2341–2368, 2013.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *International Conference on Machine Learning*, 2018.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.

Amanpreet Singh, Tushar Jain, and Suresh Veeraraghavan. Individualized controlled continuous communication model for multiagent cooperative and competitive tasks. In *International Conference on Learning Representations*, 2019.

Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. In *Advances in Neural Information Processing Systems*, 2016.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. Non-local neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2018.

Ronald J. Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, 2020.